Plan de pruebas: Filtrar las estaciones por huecos disponibles

Aplicación Bicis · Grupo 3 · Historia de usuario 532335 · Autor: Diego Alonso

Historia de usuario

Filtrar las estaciones por huecos disponibles (ID 532335). Yo, como visitante temporal, quiero ver solo las estaciones con un mínimo de huecos libres de manera que no llegue a una estación que esté llena.

Interfaz esperada: mock-up de la historia (Docs/Mockups/US532335-FiltrarEstacionesPorHuecos-Mockups.pdf). El diálogo de filtros que se abre con el icono del embudo de la barra superior incluye un nuevo campo "Mínimo de huecos libres", junto al de bicis libres, y el embudo indica cuántos filtros hay activos.

Niveles de prueba

Los niveles de prueba que se van a aplicar son los siguientes:

- **Pruebas de aceptación.** Son los criterios de aceptación acordados con el Product Owner. Se ejecutan de forma manual en el Sprint Review sobre la versión de depuración, con los datos del Anexo 1, porque las pruebas necesitan estaciones sin dato de huecos o con un dato anómalo y la API real no lo garantiza.
- **Pruebas de interfaz.** Un caso de éxito y uno de error, implementados con Espresso abriendo la pantalla de estaciones de TUeBICI con los datos del Anexo 1.
- **Pruebas unitarias.** Se aplican a los métodos nuevos de StationFilter (modelo) y StationsPresenter, con técnicas de caja negra (partición equivalente y análisis de valores límite), usando JUnit. En el presenter la vista y el repositorio se sustituyen por mocks de Mockito.
- **Pruebas de integración.** Se prueba StationsPresenter con el modelo y el repositorio reales (JsonNetworksRepository con los datos del Anexo 1) y solo la vista simulada con Mockito.

A continuación, se muestra una especificación detallada de los casos de prueba a aplicar en cada nivel.

Criterios de aceptación

Todas las pruebas parten de la pantalla de estaciones de TUeBICI (Santander), que se abre al pulsar TUeBICI en la lista de redes. El filtro se abre con el icono del embudo de la barra superior. Los datos concretos de cada prueba son los del Anexo 1 (Tabla 9).

Prueba 1. Caso de éxito

Contexto: la red tiene estaciones con distinto número de huecos libres, unas por encima y otras por debajo del mínimo que se va a elegir.

1. El usuario selecciona TUeBICI en la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red (42 estaciones).
3. El usuario pulsa el icono del filtro, introduce 15 en el campo "Mínimo de huecos libres" y pulsa Aplicar.
4. Se verifica que se muestran 8 estaciones ("Santander, ES · 8 estaciones") y que la primera es "Renfe" (20 huecos).
5. Se verifica que la lista está ordenada de más a menos huecos libres ("Renfe" 20, "Castilla Hermida" 19, "Correos" 19, ...) y que no aparece ninguna estación con menos de 15 (por ejemplo, "PTCAN 1", con 12 huecos).

Nota: Los huecos libres son los anclajes vacíos de la estación, donde el usuario puede dejar una bici (campo empty_slots de la API). Con el filtro de huecos activo, la lista se ordena de más a menos huecos libres y, a igualdad de huecos, primero la estación con más bicis libres, como en el mock-up (Docs/Mockups/US532335-FiltrarEstacionesPorHuecos-Mockups.pdf). Si el mínimo de huecos es 0 el filtro no se activa: se muestran todas las estaciones de la red, también las que no informan de sus huecos, en el orden habitual de la lista (acordado con el Product Owner en el Sprint Planning).

Prueba 2. Estación en el límite

Contexto: alguna estación tiene exactamente el número de huecos libres que se va a elegir ("Renfe", con 20 huecos, la que más tiene de la red).

1. El usuario selecciona TUeBICI en la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario pulsa el icono del filtro, introduce 20 y pulsa Aplicar.
4. Se verifica que la estación que está justo en el límite, "Renfe", aparece en la lista, y que es la única estación mostrada.

Prueba 3. Ninguna estación cumple el filtro

Contexto: todas las estaciones de la red tienen menos huecos libres que el mínimo que se va a elegir.

1. El usuario selecciona TUeBICI en la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario pulsa el icono del filtro, introduce 21 y pulsa Aplicar.
4. Se verifica que no aparece ninguna estación en la lista.
5. Se verifica que aparece el mensaje "No hay estaciones con los huecos libres seleccionados".
6. El usuario vuelve a abrir el filtro, cambia el valor a 15 y pulsa Aplicar. Se verifica que el mensaje desaparece y se muestran las 8 estaciones de la Prueba 1.

Prueba 4. Cambio del número de huecos

Contexto: el usuario ya ha aplicado un filtro por huecos libres sobre la lista de estaciones.

1. El usuario selecciona TUeBICI en la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario pulsa el icono del filtro, introduce 8 y pulsa Aplicar. Se muestran 24 estaciones.
4. El usuario vuelve a abrir el filtro, cambia el valor a 3 y pulsa Aplicar.
5. Se verifica que se muestran 36 estaciones, incluidas las que antes quedaban fuera, como "Nueva Montaña" (5 huecos) y "Magdalena" (4 huecos).
6. Se verifica que no aparece ninguna estación con menos de 3 huecos libres (por ejemplo, "Renfe" sí aparece y "Estación de Gamazo (NE)", con 0 huecos, no).

Prueba 5. Datos ausentes o anómalos

Contexto: alguna estación no informa de sus huecos libres o tiene un valor que no tiene sentido (en los datos de prueba, "Sector 3 (NE)" sin el dato, "San Román Corbán (NE)" con -1 y "Alisal (NE)" con el texto "sin dato").

1. El usuario selecciona TUeBICI en la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario pulsa el icono del filtro, introduce 1 y pulsa Aplicar.
4. Se verifica que las estaciones con datos correctos se filtran igual que en el caso de éxito: se muestran 36 estaciones y no aparecen las tres que tienen 0 huecos.
5. Se verifica que "Sector 3 (NE)", "San Román Corbán (NE)" y "Alisal (NE)" no aparecen en la lista.

Nota: Una estación sin dato de huecos, o con un dato anómalo, no se muestra con el filtro activo porque no se puede garantizar que tenga sitio para dejar la bici. Los dos filtros son independientes: "Plaza Pombo (NE)", que no informa de sus bicis pero tiene 7 huecos, sí aparece.

Pruebas de interfaz

Se implementan con Espresso en el directorio androidTest, en el paquete es.unican.bicis.activities.stations, que es el de StationsView, la vista que lanza los escenarios. Cada clase se nombra con el criterio de aceptación que implementa (<CriterioAceptación>UITest). Abren StationsView con la red TUeBICI y los datos del Anexo 1. Hay un caso de éxito y uno de error.

Tabla 1. Pruebas de interfaz

ID	Criterio	Implementación y entrada	Resultado esperado
UI.1	Prueba 1. Caso de éxito	FiltrarEstacionesPorHuecosExitoUITest. Mínimo de huecos 15	8 estaciones ("Santander, ES · 8 estaciones"), la primera "Renfe" (20 huecos), ordenadas de más a menos huecos; no aparece "PTCAN 1"
UI.2	Prueba 3. Ninguna estación cumple el filtro	FiltrarEstacionesPorHuecosNingunaCumpleUITest. Mínimo de huecos 21	Mensaje "No hay estaciones con los huecos libres seleccionados" en lugar de la lista

Las pruebas 2, 4 y 5 quedan cubiertas por las pruebas unitarias del modelo (Tabla 3) y por las de integración (Tabla 6), que usan los mismos datos.

Métodos a probar

Métodos nuevos o modificados para esta historia de usuario. Los nombres del presenter y de la vista son los del contrato de la historia (IStationsContract, tarea 978253).

Tabla 2. Métodos a probar

Método	Qué hace	Pruebas
StationFilter (modelo) filterByMinEmptySlots(stations, minEmptySlots)	Devuelve las estaciones con un número de huecos libres igual o mayor que el mínimo, en el mismo orden. Excluye las que no tienen el dato o lo tienen anómalo. Con mínimo 0 devuelve todas. Precondiciones: la lista no es null y el mínimo es mayor o igual que 0 (las garantiza StationsPresenter).	Unitarias (Tabla 3), integración (Tabla 6) e interfaz (UI.1, UI.2)
StationFilter (modelo) hasValidEmptySlots(station)	Indica si una estación informa de un número de huecos válido (no null y no negativo).	Unitarias (Tabla 4)
StationsPresenter onMinEmptySlotsFilterApplied(text)	Se llama al aplicar el filtro de huecos. Valida el valor, filtra (combinando con el filtro de bicis), ordena el resultado de más a menos huecos (a igualdad, más bicis primero) y pide a la vista que muestre las estaciones, el mensaje de sin resultados o el error de valor no válido. Con 0 muestra todas las estaciones.	Unitarias (Tabla 5), integración (Tabla 6) e interfaz

Método	Qué hace	Pruebas
StationsView showNoStationsWithMinEmptySlots(min) showInvalidMinEmptySlots() showMinEmptySlotsFilterActive(min) showActiveFiltersCount(numActiveFilters)	Muestran el mensaje sin resultados, el error del campo, el mínimo de huecos activo y el número de filtros activos en el embudo.	Interfaz (UI.1, UI.2)

Pruebas unitarias

Pruebas unitarias de la clase StationFilter

Se aplican técnicas de clases de equivalencia y AVL para la definición de los casos de prueba. Además, se deberá garantizar la cobertura de sentencias y de condición/decisión. Salvo que se indique otra cosa, la lista de partida, que llamaremos L, contiene cinco estaciones con 0, 4, 7, sin dato (null) y -3 huecos libres, en ese orden. Implementadas en StationFilterTest (directorio test, paquete es.unican.bicis.model). En filterByMinEmptySlots no se prueban la lista null ni los mínimos negativos, porque incumplen sus precondiciones: es StationsPresenter quien valida el valor introducido por el usuario y solo llama al modelo con la lista de estaciones ya cargada. Las estaciones sin dato o con un dato anómalo sí se prueban, porque son datos reales de la API.

Método filterByMinEmptySlots(stations, minEmptySlots)

Tabla 3. Casos de prueba del método filterByMinEmptySlots

Identificador	Entrada	Valor esperado
UFH.1a	L, minEmptySlots = 4 (Valor intermedio, igual al de una estación)	[4, 7]
UFH.1b	L, minEmptySlots = 7 (Igual al máximo: valor límite)	[7]
UFH.1c	L, minEmptySlots = 8 (Justo por encima del máximo: valor límite)	[] (lista vacía)
UFH.1d	L, minEmptySlots = 1 (Menor valor que pide el usuario)	[4, 7]
UFH.1e	L, minEmptySlots = 0 (Valor 0: se muestran todas)	[0, 4, 7, sin dato, -3] (L completa, en el mismo orden)
UFH.1g	stations = [], minEmptySlots = 1	[]
UFH.1i	stations = [sin dato, -3, estación null], minEmptySlots = 1 (Solo estaciones sin dato válido)	[]
UFH.1j	L en orden [7, 0, 4], minEmptySlots = 4	[7, 4] (se mantiene el orden) y L no se modifica

Método hasValidEmptySlots(station)

Tabla 4. Casos de prueba del método hasValidEmptySlots

Identificador	Entrada	Valor esperado
UFH.2a	Estación con 0 huecos (valor límite)	true

Identificador	Entrada	Valor esperado
UFH.2b	Estación con 7 huecos	true
UFH.2c	Estación sin dato (null)	false
UFH.2d	Estación con -3 huecos	false
UFH.2e	station = null	false

Pruebas unitarias del presenter StationsPresenter

Se prueba el presenter aislado. La vista (IStationsContract.View) y el repositorio (INetworksRepository) se sustituyen por mocks de Mockito; el repositorio responde con las estaciones indicadas. Se comprueba qué métodos de la vista llama el presenter en cada caso. La red es TUeBICI (Santander, ES) y, salvo que se indique otra cosa, sus estaciones tienen 0, 4, 7 y sin dato huecos libres. Implementadas en StationsPresenterTest (directorio test, paquete es.unican.bicis.activities.stations).

Método onMinEmptySlotsFilterApplied(minEmptySlotsText)

Tabla 5. Casos de prueba del método onMinEmptySlotsFilterApplied

Identificador	Entrada	Valor esperado
UPH.1a	"4"	showStations([7, 4], ordenadas de más a menos huecos) y el embudo indica 1 filtro activo
UPH.1b	"7" (valor límite)	showStations([7])
UPH.1c	"1" (menor valor válido)	showStations([7, 4])
UPH.1d	" 7 " (con espacios)	showStations([7])
UPH.1e	"8" (justo por encima del máximo)	showNoStationsWithMinEmptySlots (8)
UPH.1f	"0" (muestra todas)	showStations con las 4 estaciones, también la que no tiene dato
UPH.1g	"-1", "abc", "2.5", "", null, "99999999999" (Valores no válidos)	showInvalidMinEmptySlots() y no se filtra ni cambia la lista
UPH.1h	"1" con estaciones [sin dato]	showNoStationsWithMinEmptySlots (1)
UPH.1i	"4" con el filtro de bicis activo (mínimo 1) y estaciones [0 bicis y 7 huecos, 2 bicis y 4 huecos]	showStations con solo la segunda (se cumplen los dos filtros) y el embudo indica 2 filtros activos

Pruebas de integración

Se prueba StationsPresenter.onMinEmptySlotsFilterApplied con el modelo real (StationFilter, Station) y el repositorio real (JsonNetworksRepository, con JUnit y Robolectric), y solo la vista simulada con Mockito. Antes de cada caso se llama a init con la red TUeBICI, que carga sus 42 estaciones de res/raw/network_tuebici.json (Anexo 1), y se verifica con Mockito lo que el presenter pide a la vista. Implementadas en StationsPresenterITest (directorio test, paquete es.unican.bicis.activities.stations), que toma el nombre del presenter por ser la clase principal de la prueba.

Tabla 6. Casos de prueba de integración del filtro por huecos

Identificador	Entrada (minEmptySlotsText)	Valor esperado
IFH.1a	"15" (caso de éxito)	showStations con 8 estaciones ordenadas de más a menos huecos: "Renfe" (20), "Castilla Hermida" (19), "Correos" (19), "Tabacalera/Varadero" (18), "Ayuntamiento (NE)" (16) e "Intercambiador Sardinero" (15) al principio
IFH.1b	"20" (estación en el límite)	showStations con solo "Renfe"
IFH.1c	"21" (ninguna estación cumple)	showNoStationsWithMinEmptySlots (21). No se llama a showStations
IFH.1d	"8" y después "3" (cambio del número de huecos)	Primero showStations con 24 estaciones; después showStations con 36, que incluye "Nueva Montaña" y "Magdalena"
IFH.1e	"1" (datos ausentes o anómalos)	showStations con 36 estaciones, sin "Sector 3 (NE)", "San Román Corbán (NE)" ni "Alisal (NE)"
IFH.1f	"0" (muestra todas)	showStations con las 42 estaciones, también "Sector 3 (NE)", "San Román Corbán (NE)" y "Alisal (NE)"

Reporte final

Resultado de la ejecución de las pruebas de la historia. Rellenar al terminar el Sprint.

Tabla 7. Resultado de las pruebas

Pruebas	Autor / ejecutor	Resultado
Unitarias StationFilter (UFH.1 – UFH.2)	Diego Alonso	Superadas (9/9)
Unitarias StationsPresenter (UPH.1)	David González	Pendiente
Integración StationsPresenter (IFH.1)	Diego Alonso	Pendiente
Interfaz UI.1 – UI.2	David González	Pendiente
Aceptación Pruebas 1 – 5	David González	Pendiente

Fallos detectados gracias a las pruebas: (rellenar al terminar el Sprint).

Anexo 1. Datos de prueba

Los datos con los que se lanza la aplicación en modo debug son los de res/raw/networks.json y res/raw/network_tuebici.json. TUeBICI tiene 42 estaciones y la que más huecos libres tiene es "Renfe" (20). Para poder probar el caso de estaciones sin dato de huecos se han modificado tres de ellas, como se muestra en la Tabla 8. Se han elegido estaciones que no se usan en las pruebas de otras historias, y su total de anclajes en la lista no cambia, porque se toma de extra.slots. La Tabla 9 indica cuántas estaciones deben verse para cada valor del filtro y cuál es la primera, con la lista ordenada de más a menos huecos.

Tabla 8. Estaciones de TUeBICI modificadas para las pruebas

Estación	empty_slots original	empty_slots en los datos de prueba
Sector 3 (NE)	4	Ausente (no informa)
San Román Corbán (NE)	3	-1 (valor negativo)
Alisal (NE)	3	"sin dato" (valor no válido)

Tabla 9. Resultado esperado del filtro por huecos sobre TUeBICI

Mínimo de huecos	Estaciones mostradas	Primera estación	Se usa en
Sin filtro	42	Nueva Montaña (5 huecos)	-
0	42 (todas)	Nueva Montaña (sin filtro, orden habitual)	IFH.1f
1	36	Renfe (20 huecos)	Prueba 5, IFH.1e
3	36	Renfe (20 huecos)	Prueba 4, IFH.1d
8	24	Renfe (20 huecos)	Prueba 4, IFH.1d
15	8	Renfe (20 huecos)	Pruebas 1 y 3, UI.1, IFH.1a
20	1	Renfe (20 huecos)	Prueba 2, IFH.1b
21	0 (mensaje sin resultados)	-	Prueba 3, UI.2, IFH.1c